Component classification and design rationale

This artifact groups the six required neural network components by their primary function. The organizing rule follows what each component contributes to a prediction or a training update. These are explanatory groups for this artifact, rather than a universal taxonomy.

Classification by primary function

Component	Functional group	Reason for placement
Layers	Architecture	Group units into input, hidden and output stages. Their order and size define the network's structure.
Neurons	Architecture	Provide the computational units within layers. Each non-input neuron combines weighted inputs and a bias, then applies an activation.
Weights	Learned parameters	Scale connections between units. Training adjusts them to change the model's predictions. Biases are additional learned parameters.
Activation functions	Nonlinear transformation	Transform a neuron's weighted input. Nonlinear functions allow a network to represent nonlinear patterns.
Loss functions	Training objective	Compare predictions with targets and assign an error value. The objective defines what training seeks to reduce.
Optimization algorithms	Parameter update	Use gradients to update weights and biases. Backpropagation calculates the gradients that the optimizer applies.

How the components were sorted

1. Choose a functional sorting rule. For each component, identify its main contribution to the forward computation or training cycle. This provides a consistent basis for grouping items that look different in a diagram.

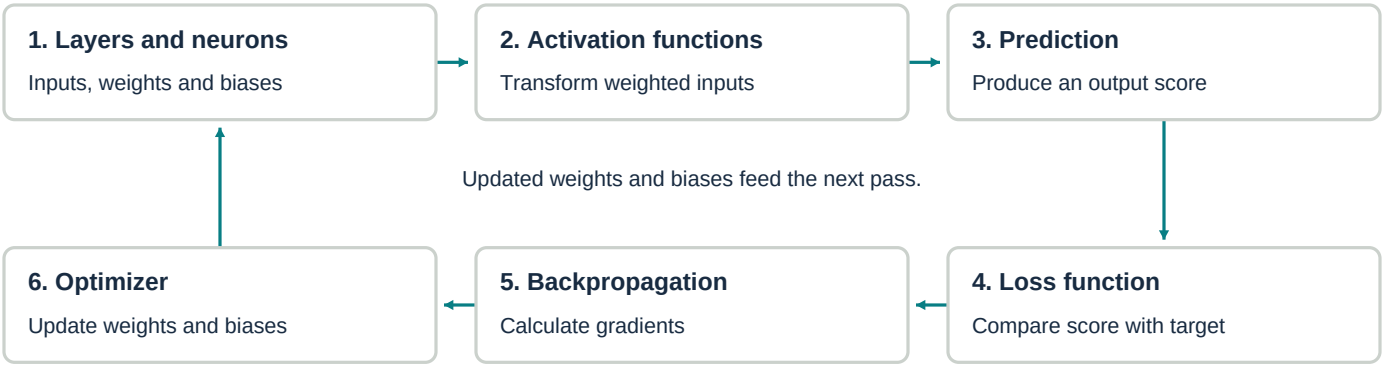
2. Resolve overlapping roles. A neuron uses weights and an activation, but those parts retain distinct roles. Weights are learned values; an activation is the transformation applied to their weighted sum. Layers describe structure, rather than a set of trainable values by themselves.

3. Separate the objective from the update rule. Loss evaluates a prediction against a target. The optimizer changes parameters to reduce that loss. Keeping these categories separate explains why backpropagation computes gradients while gradient descent applies updates.

4. Check the categories against equations and evidence. The worked neuron example connects the diagram to z = sum(w_i x_i) + b and a = f(z). Parameter counting confirms 18 weights plus 7 biases for the 2-4-2-1 network. Official Playground source confirms the demo's half-squared-error loss and classification output activation.

RELATIONSHIPS, EVIDENCE AND LEARNING

How the parts work together



During these runs, the optimizer changes learned weights and biases. Architecture and chosen activations remain fixed. Held-out test examples assess performance without driving parameter updates.

Interpreting the recorded results

The same larger spiral model's test loss decreased from 0.451 at 200 epochs to 0.032 at 1,000 epochs. This is evidence of improvement with additional training in that run. Cross-configuration comparisons are exploratory because seeds or initialization differed, and the noisy circle stopped at 217 epochs. Values displayed as 0.000 are rounded. These loss values are not accuracy percentages, and no wall-clock training time was measured.

Reflection on learning

Draft for review and personalization before submission.

Creating this artifact helped me connect neural network terminology with a model's calculations. Grouping components by function clarified the relationship between architecture, learned parameters, nonlinear transformations, the training objective, and parameter updates. The worked neuron example made the role of an activation easier to explain. Checking the dense-layer formula also reinforced why a 2-4-2-1 network has 18 weights and seven biases.

A central challenge was making fair comparisons. The circle trials used different seeds, and the noisy trial stopped at 217 epochs. My strategy was to report those settings alongside the losses and describe the comparisons as exploratory. The clearest training comparison came from continuing the same larger spiral model: test loss decreased from 0.451 at 200 epochs to 0.032 at 1,000 epochs. That observation connected model capacity and training effort to the course concept of generalization. It also showed why lower training loss alone cannot establish performance on new data.

Designing the visuals required balancing completeness with readability. I used the architecture diagram for data flow and a separate training loop for loss, gradients, and updates. This process strengthened my ability to explain how components work together rather than list definitions. In future experiments, I would use matched epoch budgets and repeat each configuration across several seeds. In professional work, these habits would help me communicate model behavior clearly and support recommendations with appropriately limited evidence.

Primary sources and full evidence

TensorFlow Playground and experiment screenshots

Playground source: neuron, error and update definitions

Playground source: output activation and training steps

Google ML Crash Course: nodes and hidden layers